

# How search ranks posts

Current implementation, with the formulas written out.

## 1. Where the tags come from

The local `content_taxonomy_3_1.tsv` contains **704 usable labels**. Rows have an ID, parent ID, name, and tier columns. The loader uses only the **ID and name**: the hierarchy is not used in scoring. For example, *Programming Languages* has ID 631 and parent 599.

The model `sentence-transformers/paraphrase-MiniLM-L3-v2` encodes each label name once when the module loads. It also encodes each search query; for a new post, its input is the title, the literal separator [SEP], and the description (when nonblank). Let  $e(x)$  be the model's embedding:

$$c_i(x) = \frac{e(x) \cdot e(\text{label}_i)}{\|e(x)\| \|e(\text{label}_i)\|}, \quad L(x) = \text{Top}_{10}\{c_i(x)\}, \quad w_i(x) = \max(0, c_i(x)).$$

Select the ten highest cosine similarities *before* clipping negatives to zero. Store each selected label's ID, name, score, and rank. These scores are similarities, **not probabilities**; they need not sum to one. No minimum similarity is required, so even weak or zero-score tags can be retained.

## 2. Query preparation and candidate retrieval

Lowercase the query; extract tokens matching `[a-z0-9+#.]+`; discard one-character tokens and the code's stop-word list; then remove duplicates. Call the remaining token set  $Q$ . The semantic tags above use the original trimmed query, not this filtered token set.

Retrieve a post if **any** query token occurs as a case-insensitive *substring* in its title or description, **or** it shares any selected query-tag ID. Duplicate database results are removed. Retrieval uses tag IDs, not tag weights. An empty query returns no results.

## 3. Text and tag relevance

For post  $p$ , let  $W(x)$  be the set of lowercased regex tokens in field  $x$ . Scoring uses *whole-token* matches, unlike the substring retrieval step:

$$m(Q, x) = \begin{cases} |Q \cap W(x)|/|Q|, & |Q| > 0, \\ 0, & |Q| = 0, \end{cases} \quad T_p = 0.70 m(Q, \text{title}_p) + 0.30 m(Q, \text{description}_p).$$

$T_p$  is the text score: the title contributes 70%, the description 30%. Repeated occurrences of a word do not increase its contribution. With query tag IDs  $L_q$  and stored post tag IDs  $L_p$ , the tag score is

$$G_p = \min \left( 1, \sum_{i \in L_q \cap L_p} w_i(q) w_i(p) \right).$$

Only identical tag IDs contribute. This is a clipped weighted overlap, not cosine similarity between the query and the post itself.

The combined relevance adds a bonus when both signals agree:

$$R_p = (0.65T_p + 0.35G_p)(1 + 0.30T_pG_p).$$

$T_p, G_p \in [0, 1]$ , but  $R_p$  can reach 1.30; it is not a probability.

## 4. Engagement and freshness

Let  $\mathcal{C}$  be the complete candidate set, *before* relevance filtering. For likes  $\ell_p$ , comment count  $c_p$ , and distinct user-view count  $v_p$ , normalize each count against the corresponding candidate maximum:

$$N(x, M) = \begin{cases} \ln(1+x)/\ln(1+M), & M > 0, \\ 0, & M = 0, \end{cases} \quad M_x = \max_{j \in \mathcal{C}} x_j,$$

$$E_p = 0.55N(\ell_p, M_\ell) + 0.30N(c_p, M_c) + 0.15N(v_p, M_v), \quad B = \text{median}_{j \in \mathcal{C}} E_j.$$

Log scaling reduces the advantage of very large raw counts. Comments include replies; views count related user-view records, not reading seconds. The median is recomputed for each search, not stored as a daily baseline.

For age  $a_p = \max(0, \text{age in days})$ , freshness has a 14-day half-life:

$$F_p = 0.5^{a_p/14}. \quad F_p = 1 \text{ at day 0, } 0.5 \text{ at day 14, } 0.25 \text{ at day 28.}$$

## 5. Eligibility and final ordering

Keep a candidate if at least one condition holds:

$$R_p \geq 0.05 \quad \vee \quad G_p \geq 0.10 \quad \vee \quad T_p > 0.$$

The last condition reflects `ALLOW_ANY_TEXT_MATCH = True`. Popularity modifies the relevance/freshness blend:

$$P_p = \max(0.30, 1 + E_p - B), \quad S_p = (0.70R_p + 0.30F_p)P_p.$$

Return eligible posts in descending  $S_p$  order. A post at median engagement has  $P_p = 1$ ; above-median engagement boosts it, below-median engagement reduces it. The 0.30 floor limits that reduction. No explicit database tie-breaker is specified; Python's stable sort preserves the incoming order for equal scores.

## 6. Worked example

Suppose  $Q = \{\text{python}, \text{tutorial}\}$ . The title matches both tokens, and the description matches only one. Suppose the shared tag weights are (0.8, 0.5) for the query and (0.7, 0.4) for the post. Then

$$T_p = 0.70(1) + 0.30(1/2) = 0.85, \quad G_p = 0.8(0.7) + 0.5(0.4) = 0.76,$$

$$R_p = (0.65(0.85) + 0.35(0.76))(1 + 0.30(0.85)(0.76)) \approx 0.9771.$$

For a 14-day-old post with  $E_p = 0.60$  and candidate median  $B = 0.40$ :

$$F_p = 0.50, \quad P_p = 1.20, \quad S_p = (0.70(0.9771) + 0.30(0.50))(1.20) \approx \boxed{1.0008}.$$

The post passes the relevance gate. Final scores can exceed one.

## Implementation notes

This is a hand-weighted ranking formula, not a learned ranking model. Candidate maxima and the median include posts later filtered out, so changing the candidate pool can change an existing post's score. Substring retrieval can admit posts with zero whole-token matches; tags or relevance must then pass the gate. No stemming, spelling correction, or phrase-matching stage is implemented.

**Source of truth:** `home/scoring.py` (weights and ranking), `home/views.py` (search orchestration), `threads/label_classifier.py` (tag assignment), `threads/views.py` (new-post classification), and `model/tags/content_taxonomy_3_1.tsv` (label vocabulary).